



OPEN Improved multi-strategy artificial rabbits optimization for solving global optimization problems

Ruitong Wang^{1,3}✉, Shuishan Zhang^{1,3} & Bo Jin²

Artificial rabbits optimization (ARO) is a metaheuristic algorithm based on the survival strategy of rabbits proposed in 2022. ARO has favorable optimization performance, but it still has some shortcomings, such as weak exploitation capacity, easy to fall into local optima, and serious decline of population diversity at the later stage. In order to solve these problems, we propose an improved multi-strategy artificial rabbits optimization, called IMARO, based on ARO algorithm. In this paper, a roulette fitness distance balanced hiding strategy is proposed so that rabbits can find better locations to hide more reasonably. Meanwhile, in order to improve the deficiency of ARO which is easy to fall into local optimum, an improved non-monopoly search strategy based on Gaussian and Cauchy operators is designed to improve the ability of the algorithm to obtain the global optimal solution. Finally, a covariance restart strategy is designed to improve population diversity when the exploitation is stagnant and to improve the convergence accuracy and convergence speed of ARO. The performance of IMARO is verified by comparing original ARO algorithm with six basic algorithms and seven improved algorithms. The results of CEC2014, CEC2017, CEC2022 show that IMARO has a good exploitation and exploration ability and can effectively get rid of local optimum. Moreover, IMARO produces optimal results on six real-world engineering problems, further demonstrating its efficiency in solving real-world optimization challenges.

Keywords Artificial rabbit optimization, Roulette fitness distance balanced hiding strategy, Non-monopoly search strategy, Covariance restart strategy, CEC2014, CEC2017, CEC2022

Artificial Intelligence technology has made remarkable progress in the past few years, not only making breakthroughs in technical performance and application scope, but also playing an important role in promoting the development and transformation of various industries^{1,2}. The optimization problem is a core component of AI technology. Solving optimization problems in engineering and scientific research is a specialized class of mathematical problems. These mathematical problems aim to find the best solution from multiple alternatives to maximize or minimize the target. Real-world applications often need to impose multiple design constraints in order to meet solution availability requirements. Most of these problems are nonlinear, complex, and non-microscopic, and thus difficult to solve. To tackle these optimization problems, researchers have presented a variety of efficient and useful algorithms. While traditional gradient-based deterministic algorithms achieve faster processing speeds, they easily fall into local optima as the problem size increases³. In addition, the performance of deterministic algorithms largely depends on the characteristics of problem and initial parameter values. Metaheuristic algorithms, as a class of stochastic algorithms, solve highly complex optimization problems by manipulating one or more potential solutions to reach the optimal or most favorable solution. These algorithms do not depend on problem characteristics, have a simple structure, are highly flexible, do not rely on gradient information, and have been found to be more effective than algorithms based on gradient information in generating optimal or near-optimal solutions for indivisible rough optimization⁴. Therefore, they have received extensive attention and flourished among scholars. In addition to classical numerical optimization, metaheuristic algorithms have demonstrated their effectiveness in a wide range of optimization tasks including, but not limited to, feature selection^{5,6}, traveling sales problems^{7,8}, image segmentation problems^{9,10}, wireless sensor coverage problems^{11,12}, mission planning^{13,14}. In the past decades, new metaheuristic algorithms based on evolutionary

¹Leicester Institution, Dalian University of Technology, Dalian 124221, China. ²Department of Electrical and Computer Engineering (DEEC), Institute of Systems and Robotics (ISR), University of Coimbra, 3030-290 Coimbra, Portugal. ³These authors contributed equally: Ruitong Wang and Shuishan Zhang. ✉email: 1160827774@mail.dlut.edu.cn

theory, laws of physics, biological group behavior and human social behavior have been proposed, and Table 1 lists some of the algorithms proposed in recent years.

Artificial rabbits optimization is a swarm-based metaheuristic algorithm inspired by the survival strategies of rabbits in nature, proposed by Wang et al.⁴⁰. ARO has been applied in several fields since its proposal^{41–43}. ARO achieves the problem solution through execution of meandering foraging strategy and random hiding strategy. From the design principle of the algorithm, the communication between ARO populations is insufficient and the convergence efficiency is low in exploitation phase, thus ARO still has potential for improvement. In addition, according to No Free Lunch (NFL) theorem⁴⁴, no class of algorithms can solve all problems well, which also reflects that the algorithm is scalable.

Cao et al.⁴⁵ used an adaptive weight adjustment strategy to enhance ARO and combined it with support vector machines to classify and recognize vibration signals. Abd et al.⁴⁶ utilized Gaussian variants and crossover operators to improve the performance of ARO. Ladan et al.⁴⁷ used a diagonally homogeneous initialization process to improve the convergence of ARO. Hu⁴⁸ proposed the IARO algorithm, which improves population diversity through a mutation strategy, an adaptive population strategy, and an elite feedback strategy. Wang et al.⁴⁹ developed an ARO variant by combining Levy flight and selective opposition strategies. Nehmedo et al.⁴² proposed a quantum mechanical artificial rabbit optimization, for solving energy management problems. The above-mentioned literatures have examined ARO and proposed their respective improvement methods. These literatures target switching between exploitation and exploration, population initialization, and enhancing population diversity to boost ARO performance. These improvements include aspects of ARO, but a single ARO variant does not have them all included. In addition, the NFL theorem encourages us to continuously improve existing algorithms.

To address the shortcomings of ARO, this paper proposes an improved multi-strategy artificial rabbits optimization (IMARO). In the first step, to address problem of low exploitation ability in the stochastic hiding phase, this paper proposes a roulette fitness distance balanced hiding strategy, so that ARO can have better local search ability and a certain degree of exploration ability in the exploitation phase. Second, improved non-monopoly search strategy is proposed to adjust the optimal agent and enhance local optimal avoidance ability of ARO. Finally, a covariance restart strategy is used to restart the algorithm when it stagnates, which improves population diversity and convergence performance by enhancing the information exchange between individuals in the population.

In the experimental section, the performance of IMARO was validated using the CEC2014, CEC2017 and CEC2022 test suites. The superiority of the methodology of this paper is illustrated by numerical analysis, convergence analysis, Friedman test and Wilcoxon rank sum test. The main contributions of this paper are as follows:

Evolution-based algorithms	Genetic Algorithm (GA) ¹⁵
	Differential Evolution (DE) ¹⁶
	Genetic Programming (GP) ¹⁷
	Evolutionary Strategy (ES) ¹⁸
Physics-based algorithms	Simulated Annealing (SA) ¹⁹
	Gravitational Search Algorithm (GSA) ²⁰
	Sine Cosine Algorithm (SCA) ²¹
	Multi-Verse Optimization (MVO) ²²
	Henry Gas Solubility Optimization (HGSO) ²³
	Snow Ablation Optimizer (SAO) ²⁴
Human-based algorithms	Archimedes optimization algorithm(AOA) ²⁵
	Teaching And Learning Based Optimization (TLBO) ²⁶
	Social Network Search (SNS) ²⁷
	Group Teaching Optimization Algorithm (GTOA) ²⁸
	Population-based Vortex Search Algorithm (PVSA) ²⁹
	Running City Game Optimizer (RCGO) ³⁰
Swarm-based algorithms	Social Evolution And Learning Optimization (SELO) ³¹
	Particle Swarm Optimization (PSO) ³²
	Ant Colony Optimization (ACO) ³³
	Whale Optimization Algorithm (WOA) ³⁴
	Grey Wolf Optimization (GWO) ³⁵
	Remora Optimization Algorithm (ROA) ³⁶
	Reptile Search Algorithm (RSA) ³⁷
	Tuna Swarm Optimization (TSO) ³⁸
	Sand Cat Swarm Optimization (SCSO) ³⁹

Table 1. Metaheuristic algorithms.

- (1) The roulette fitness distance balanced hiding strategy (RHS) is proposed to adjust the original ARO's stochastic hiding strategy, which improves exploitation ability of the algorithm and retains a certain degree of exploration ability, effectively helping the algorithm to jump out of the local optimum.
- (2) The introduction of improved non-monopoly search strategy (INSS) to perturb the updated optimal solution further improves the ability of the algorithm to obtain the global optimal solution by combining Gaussian and Cauchy operators.
- (3) The covariance restart strategy (CRS) is used to assist the algorithm to restart when it stagnates, which enriches the population diversity and enhances quality and search efficiency of the agents by reinforcing the information exchange between agents in the populations.
- (4) IMARO is tested on 30 functions of CEC2014, 29 functions of CEC2017 and 12 functions of CEC2022 with different dimensions. The optimal parameter settings for improvement strategies and the effectiveness of different strategies are also discussed.
- (5) The ability of IMARO to solve real-world problems is further validated on six engineering constrained optimization problems.

The rest of the paper is organized as follows. “Artificial rabbits optimization (ARO)” section briefly introduces the ARO algorithm. “Improved multi-strategy artificial rabbits optimization (IMARO)” section presents the improved strategies of IMARO. In “Experimental results and discussion” and “Mechanical engineering problems” sections, we conducted comprehensive simulation experiments and conducted a detailed analysis of the results. Finally, “Conclusions” section summarizes this study and discusses future prospects.

Artificial rabbits optimization (ARO)

Artificial rabbits optimization is a new swarm-based meta-heuristic algorithm proposed in 2022, which is inspired by survival strategy of rabbits. The initialization phase, detour foraging strategy, random hiding strategy and switching mechanism of ARO will be introduced in following sections in turn. The pseudo-code for ARO is shown in Algorithm 1.

```

1 : Initialization :  $X_i(i=1,2,3,\dots,Np)$ ,  $t = 0$ ,  $t_{max}$ 
2 : while ( $t < t_{max}$ ) do
3 :   for every individual, do
4 :     Select guide rabbits from  $X$ 
5 :     Calculate energy factor  $A$ 
6 :     if  $A > 1$ 
7 :       Update the  $X$  using detour foraging strategy
8 :     else
9 :       Update the  $X$  using random hiding strategy
10 :    end
11 :  end for
12 :  Update population based on the fitness value of rabbits
16 : end while
17 : Return  $X_b$ 

```

Algorithm 1 Artificial rabbits optimization (ARO)

Initialization

In initial phase of ARO, as in other swarm-based algorithms, there are Np agents randomly generated in the problem space, as shown in Eq. (1).

$$X_i = lb + rand \times (ub - lb), i = 1, 2, \dots, Np \quad (1)$$

where X_i is the i th agent. lb and ub denote the lower and upper boundaries of problem space. Np denotes the number of populations. $rand$ represents a random number between 0 and 1.

Detour foraging strategy (exploration phase)

In detour foraging strategy, rabbits mainly randomly choose food away from their area to feed. The mathematical model of this strategy is represented as follows.

$$X_i^{t+1} = X_r^t + R \times (X_i^t - X_r^t) + round(0.5 \times (0.05 + r_1)) \times randn \quad (2)$$

$$R = c \times \left(e - e^{\left(\frac{t-1}{t_{max}} \right)^2} \right) \times \sin(2\pi r_2) \quad (3)$$

$$c(k) = \begin{cases} 1, & \text{if } k == \text{randperm}(D) \\ 0, & \text{else} \end{cases}, \quad k = 1, 2, \dots, D \text{ and } D = 1, 2, \dots, \lceil r_3, D \rceil \quad (4)$$

where X_i^{t+1} denotes updated agent under current iteration. X_i^t denotes a randomly selected another agent. r_1, r_2 and r_3 are randomly generated from 0 to 1. $\lceil \cdot \rceil$ symbolizes the ceil function. $t_{\max}$ denotes the maximum number of iterations. D denotes the dimension of the problem. randn is a random number that follows a normal distribution.

Random hiding strategy (exploitation phase)

During exploitation phase of ARO, rabbits avoided capture by digging holes around their burrows and hiding. The mathematical model of this random hiding is represented as follows:

$$X_i^{t+1} = X_i^t + R \times (r_4 \times b_r - X_i^t) \quad (5)$$

$$b_r = X_i^t + \left(\frac{t_{\max} - t + 1}{t_{\max}} \right) \times r_4 \times g \times X_i^t \quad (6)$$

$$g(k) = \begin{cases} 1, & \text{if } k = \lceil r_5, D \rceil \\ 0, & \text{else} \end{cases}, \quad k = 1, 2, \dots, D \quad (7)$$

where r_4 and r_5 are randomly generated from 0 to 1. After executing detour foraging strategy and the random hiding strategy, ARO will choose either the parent agent or child agent based on greedy strategy.

Transition mechanism

In ARO, rabbits will engage in a detour foraging strategy in the early stage and choose a random hiding strategy in later stage. Therefore, ARO employs the energy factor A to switch between the two strategies. When $A > 1$, the rabbit has high energy and chooses the meandering foraging strategy. On the contrary, the rabbit does not have enough energy to move widely, so it chooses the random hiding strategy. The energy factor A is calculated as follows:

$$A = 4 \times \left(1 - \frac{t}{t_{\max}} \right) \times \ln \left(\frac{1}{\text{rand}} \right) \quad (8)$$

where $t_{\max}$ denotes the max iteration number.

Improved multi-strategy artificial rabbits optimization (IMARO)

In this section, the proposed IMARO is introduced in detail. To overcome the shortcomings of the basic ARO and improve its exploitation performance, the roulette adaptive distance balanced hiding strategy is proposed to adjust the original ARO stochastic hiding strategy. This improves the exploitation capability of the algorithm, retains some exploration capability, and effectively helps the algorithm to jump out of the local optimum. Moreover, an improved non-monopoly search strategy is introduced to perturb the updating optimal solution, which further improves the algorithm's ability to obtain the global optimal solution by combining Gaussian and Cauchy operators. The covariance restart strategy is used to assist the algorithm to restart when stagnation occurs, which enriches the population diversity and improves the quality of intelligences and search efficiency by enhancing the exchange of information among the intelligences in the population. The mathematical model of IMARO is described as follows.

Roulette fitness distance balanced hiding strategy

The update method of random hidden phase is mainly used to further search the problem space, but the original formula emphasizes too much on random selection, which weakens exploitation ability. Therefore, this paper proposes a fitness distance balanced hiding strategy, which selects agents for the hiding phase by considering fitness and distance for reference. To ensure sufficient exploitation performance and some global exploration performance in randomized hiding phase, the agents are further filtered using roulette to ensure that each agent has a chance to be selected. This results in a good synchronization of the exploitation and exploration performance. The enhanced randomized hidden phase formulation is shown in Eq. (10).

$$X(\text{Score}) = (\text{norm}(\text{Fitness}) + \text{norm}(\text{Distance}))/2 \quad (9)$$

$$X_{RHS} = \text{Select}[X]_{\text{roulette}} \quad (10)$$

where $\text{norm}(\text{Fitness})$ is the normalized value of fitness and $\text{norm}(\text{Distance})$ is the normalized value of distance. X_{RHS} is the agent selected using this strategy. In this paper, based on RHS, 10 different random hidden update formulas are proposed. In the subsequent experiments, we will present these 10 different random hiding strategies and choose the best method.

Improved non-monopoly search strategy

The quality of optimal agent plays a key role in the performance of ARO, if the optimal agent falls into local optimality, it will easily cause other agents to fall into local optimality as well. In order to avoid premature

convergence, the optimal agent needs to be adjusted. The non-monopoly search strategy is a new local search method that modifies the dimension of the current solution space along the search space to further improve the quality of the optimal solution. In this paper, we utilize the non-monopoly search strategy to perturb the optimal agent to further improve performance. The non-monopoly search strategy is formulated as follows.

$$X_{bnew}(j) = rand \times X_b(random) \quad (11)$$

$$X_{bnew}(j) = X_b(j) - (X_b(IN) \times rand) \times eps - (X_b(j) - 1) \quad (12)$$

where $X_{bnew}(j)$ is dimension of the new solution. $X_b(random)$ is the random dimension of the best solution. In this strategy, Eq. (11) is used in the preperiodic, and Eq. (12) is used in the post period. In order to further improve the global search ability in the preperiodic and the local search ability in the post period. Cauchy operator and Gaussian operator are introduced in this paper. The Cauchy operator can effectively provide the agent with a wide range of perturbations, while Gaussian operator can provide agent with finer adjustments. We combine several improved algorithms using these two operators as described below.

$$X_{Gnew}(j) = Cauchy \times X_G(random) \quad (13)$$

$$X_{bnew}(j) = X_b(j) - (X_b(IN) \times rand) \times eps - Gaussian \times (X_b(j) - 1) \quad (14)$$

Covariance restart strategy

Since ARO algorithm tends to fall into local optimum at later iterations, this paper proposes a covariance restart strategy. The strategy is executed based on the current fitness of the agent and the fitness of the parent. When initializing the algorithm, an accumulator counter R_s is defined for the restart strategy with an initial value of 0. If the current fitness is less than the fitness of the parent, which indicates that there is no error in the evolution of the individual, R_s is unchanged, otherwise, R_s is added 1. When the counter value is greater than a specific threshold (defined as RT in this algorithm), the covariance restart strategy will be triggered. This strategy utilizes the weighted position of dominant population, the global optimal agent, and the current agent to adjust the agent that triggers the restart strategy. This can correctly guide agent evolution and improve algorithmic diversity. The specific formula is expressed as follows:

$$X_{new} = (X_b + X_{mean} + X)/3 + y, y \sim N(0, Cov) \quad (15)$$

$$Cov(i) = \frac{1}{0.5Np} \sum_{i=1}^{0.5Np} (X - X_{mean}) \times (X - X_{mean})^T \quad (16)$$

$$X_{mean} = \sum_{i=1}^{0.5Np} \omega_i \times X_i \quad (17)$$

$$\omega_i = \frac{\ln(0.5Np + 0.5) - \ln(i)}{\sum_{i=1}^{NP/2} (\ln(0.5Np + 0.5) - \ln(i))} \quad (18)$$

where X_{mean} is obtained from a weighted average of the best part of the agents in the rabbits according to their performance, which can represent evolutionary trend of the rabbits. The CRS mainly utilizes the overall distribution information of the dominant agents, which is not conducive to execution of CRS when the population size is small. Therefore, this paper further introduces an external archiving mechanism to put the portion of agents in each iteration into archive. When the number of archived agents is large, the redundant agents are removed based on the first-in-first-out principle. The number of archives will be confirmed by testing in experimental phase.

Implementation steps of the IMARO

The pseudo-code and flow-chart for IMARO are shown in Algorithm 2 and Fig. 1.

```

1 : Initialization :  $X_i(i=1,2,3,\dots,Np), t = 0, t_{max}$ 
2 : while ( $t < t_{max}$ ) do
3 :   for every individual, do
4 :     Calculate energy factor  $A$ 
5 :     if  $A > 1$ 
6 :       Select guide rabbits from  $X$  using random selection
7 :       Update the  $X$  using detour foraging strategy
8 :     else
9 :       Select guide rabbits from  $X$  using RHS based on Eq. (10)
10 :      Update the  $X$  using random hiding strategy
11 :    end
12 :    Update the  $X$  using CRS based on Eq. (15)
13 :  end for
14 :  Update population based on the fitness value of rabbits
15 :  Update  $X_b$  using INSS based on Eq. (13) and Eq. (14)
16 : end while
17 : Return  $X_b$ 

```

Algorithm 2 Improved multi-strategy artificial rabbits optimization (IMARO)

Computational time complexity

The time complexity reflects the processing length needed for an algorithm to resolve a problem when its scale is increasing. As for the ARO, population is Np , dimension is D , and maximum number of iterations is T . During initialization, the time complexity for fitness calculation is $O(Np \times D)$. The individual position update involves updating the positions of each individual over T iterations, so the time complexity is $O(T \times Np \times D)$. Therefore, the total time complexity of ARO is $O(Np \times D + T \times Np \times D)$. Removing lower-order terms, the overall time complexity of ARO can be simplified as $O(T \times Np \times D)$.

For IMARO, the initialization process is also $O(Np \times D)$. The time complexity of random hiding strategy with RHS is $O(T \times Np/2 \times D)$. Detour foraging time complexity is $O(T \times Np/2 \times D)$. Improved non-monopoly search strategy time complexity is $O(T \times D)$. For the covariance restart strategy, we assume that there are Nc agents executing this strategy, so the time complexity is $O(T \times Nc \times D)$. Thus, the total complexity of IMARO is as below.

$$\begin{aligned}
 O(\text{IMARO}) &= O(Np \times D + T \times Np/2 \times D + T \times Np/2 \times D + T \times D + T \times Nc \times D) \\
 &\cong O(T \times D \times (Np + Nc))
 \end{aligned}$$

Although the time complexity of IMARO is larger than that of ARO, the IMARO performance is significantly enhanced compared with that of ARO, so this case can be accepted.

Experimental results and discussion

This section conducts experiments on proposed IMARO using CEC2014 test suite, CEC2017 test suite and CEC2022 test suite for validation. The experimental results are presented in 3 subsections. “[Parameter setting and experimental environment](#)” section provides a description of experimental environment and basic and improved algorithms involved in the comparison. The improved strategies are analyzed in “[Analysis of improvement strategies](#)” section, which identifies the optimal parameters for the covariance restart strategy and the most successful roulette fitness distance balanced hiding strategy. This section also evaluates effectiveness of different strategies. In “[Comparison with basic algorithms](#)” and “[Comparison with improved algorithms](#)” sections the performance of the proposed IMARO is examined using the basic algorithm and the improved algorithm, respectively. More information on CEC 2014 test suite, CEC 2017 test set and CEC 2022 test suite can be found in references^{50–52}.

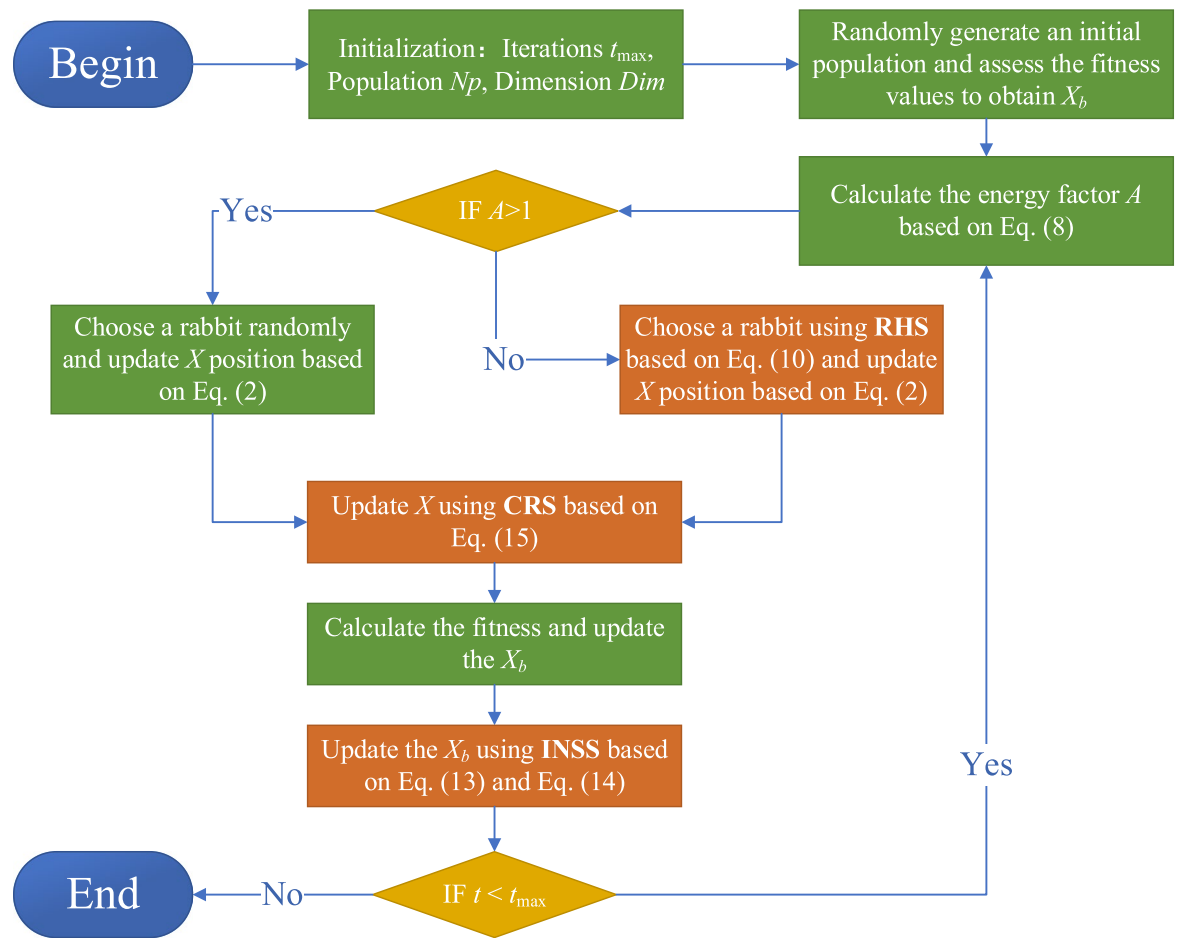


Figure 1. The flow chart of the IMARO.

Parameter setting and experimental environment

The hardware and software used for tests in our research are shown in Table 2. The experimental parameters of this paper are set as follows: the number of populations N_p is 50, the maximum number of iterations $t_{\max}$ is 300, each algorithm is run independently for 30 times, and the mean, best value and standard deviation of each run are selected and recorded. In this paper, six basic algorithms and seven improved algorithms are selected for comparison. Algorithm parameter settings are shown in Table 3.

In this paper, there are two nonparametric statistical hypothesis tests to analyze the results derived from IMARO and 14 competitors from a statistical point of view. First, Wilcoxon rank sum test is used to test the difference between IMARO and all competitors for each test function. Based on results of Wilcoxon rank sum test, it is possible to know whether there is a significant difference between performance of IMARO algorithm and each competitor algorithm on each function, and then to determine which algorithm has better performance. The Friedman test is then used to give scores of algorithms involved in the experiment and these scores represent ranking of each algorithm.

Analysis of improvement strategies

Parameter analysis of the covariance restart strategy

In the covariance restart strategy, there are two parameters that affect performance of the strategy. The first parameter is restarting threshold RT , and the other is external archive size. The restart threshold setting determines how often the CRS is enabled. The size of external archive determines how much population history information can be retained. In this paper, the optimal value of RT is chosen from $RT = \{0.2, 0.4, 0.6, 0.8, 1.0\} \times Dim$, and the optimal value of the size of the external archive AS is chosen from $AS = \{5, 10, 15\} \times Dim$. Both parameters of the CRS are related to the dimensionality of the problem. This setting allows the algorithm to adaptively adjust parameter values when dealing with problems of different dimensions, thus reducing the parameter sensitivity of the algorithm and avoiding additional parameter tuning efforts. This section compares the optimization results of covariance restart strategy under the 15 different parameter settings mentioned above using three test suites under different dimensions. Due to large size of the test data, instead of making specific presentation of the data results, the simulation results under different parameter settings are analyzed by the Friedman test to reveal the differences.

The results of this section of experiment are recorded in Tables 4, 5 and 6 and visually shown in Figs. 2, 3 and 4. It can be seen that most of the best results are accompanied by the parameter taking the value $RT = 0.2Dim$ when the external archive AS takes different values, and the ranking gets progressively worse as value of RT

increases. This indicates that the lower the restart threshold is set, the more it can improve performance of the algorithm. It is worth noting that ARO with the introduction of CRS outperforms the basic ARO in most cases, regardless of the parameter choice. According to the results in Tables 4, 5 and 6, ARO is ranked in the last place in most cases. On the other hand, for different values of external archives, we find that the performance of ARO is enhanced when the number of external archives increases. This indicates that when the number of external archives is larger, it can fully preserve historical information of good agents, which is conducive to enriching population diversity. Combining all rankings, IMARO gets the best results when $AS = 15Dim$ and $RT = 0.2Dim$, with an average ranking of 3.89. Therefore, in order to fully show the performance of IMARO, we set the external archive AS and restart threshold RT to $15Dim$ and $0.2Dim$ in this paper.

Matching analysis of the roulette fitness distance balanced hiding strategy

In this section, we evaluate the performance of different roulette fitness distance balanced hiding strategies. Table 7 gives 10 different versions. These 10 variants and the ARO algorithm are run 30 times in different dimensions of the three test suites. Since the experiments yielded a total of $11 \times 59 \times 30 \times 4 + 11 \times 12 \times 30 \times 2 = 85,800$ sets of data, they were analyzed using the nonparametric Friedman test.

The results of Friedman test for 10 variants using RHS given in Table 7 and ARO on three different test suites are shown in Table 8. The test results of all algorithms in different dimensions are used to calculate the Friedman scores. In Table 8, the best score for each case is shown in bold. In addition, the average scores for different test suites are also presented in Table 8. As can be seen from Table 8, the ARO-RHS variants proposed herein all outperform the basic ARO in general, with ARO-RHS7 having the best performance. Specifically, ARO-RHS6, ARO-RHS8, and ARO-RHS10 each achieved the best ranking once. ARO-RHS4 had 2 best scores. ARO-RHS7 had the best performance in 5 out of 10 sets of experiments. On the other hand, ARO-RHS8 had the best average performance in CEC2022, with ARO-RHS7 following. The ARO-RHS7 had the best average ranking at both CEC2014 and CEC2017.

Effective analysis of each strategy

This section analyzes the impact of three improvement strategies included in IMARO on the optimization results and convergence performance of algorithms. First, six IMARO-derived algorithms, named IMARO-1, IMARO-2, IMARO-3, IMARO-12, IMARO-13 and IMARO-23, are designed and showed in Table 9. Compared to IMARO, these six algorithms each contain one or two of the CRS, INSS, and RHS strategies. The performance of four algorithms as well as basic ARO is experimented using three test suites. In order to provide a statistically

	Item	Description
Software	Language	MATLAB 2022a
	Operating system	Windows 10
Hardware	Hard drive	1000 GB
	RAM	32.00 GB
	Frequency	4.50GHz
	CPU	AMD R9 7950x

Table 2. Software and hardware for all experiments.

Algorithm	Parameter setting
ARO	$k = 1$
RIME ⁵³	$w = 0.5$
MRFO ⁵⁴	$S = 2$
WOA ³⁴	$b = 0.75, a = 2, l = [-1, 1]$
DBO ⁵⁵	$p = 0.2$
SMA ⁵⁶	$z = 0.03$
GJO ⁵⁷	$E_1 = 1.5, c_1 = 1.5$
AFDBARO ⁵⁸	$k = 1$
dFDBARO ⁴¹	$k = 1$
DTSMA ⁵⁹	$z = 0.03, q = 0.9$
RCLMAOA ⁶⁰	$z = 0.03, \mu = 0.499, \alpha = 5$
IHAOAVOA ⁶¹	$L_1 = 0.8, \omega = 2.5, P_2 = 0.4, U = 0.00565, r = 10$
MSCSO ⁶²	$C = 0.35, SM = 2, \beta = [0, 2\pi]$
RLTLBO ⁶³	$u = 0.6, s = 1, d = 2, sgm = 0.6, lbd = 0.5$

Table 3. Parameter settings of each algorithm.

Test suite	Dim	ARO	RT=0.2Dim	RT=0.4Dim	RT=0.6Dim	RT=0.8Dim	RT=1.0Dim
CEC2014	10	12.38	10.59	10.28	11.24	10.03	10.41
	30	12.45	5.72	7.48	9.10	10.62	11.52
	50	12.41	4.10	6.17	9.48	10.55	12.45
	100	12.21	3.00	4.76	7.38	10.14	11.48
CEC2017	10	11.52	11.69	11.31	10.17	9.79	10.34
	30	14.90	3.76	7.41	8.41	12.59	12.97
	50	14.93	3.10	6.10	9.24	11.21	13.14
	100	12.66	1.97	5.69	8.90	11.93	13.90
CEC2022	10	14.83	6.00	9.33	10.00	11.67	11.33
	20	12.25	9.17	11.25	10.33	10.75	10.75
Average rank		13.05	5.91	7.98	9.43	10.93	11.83

Table 4. Ranking of algorithms with different RT when AS = 5Dim.

Test suite	Dim	ARO	RT=0.2Dim	RT=0.4Dim	RT=0.6Dim	RT=0.8Dim	RT=1.0Dim
CEC2014	10	12.38	5.52	8.17	8.14	8.17	9.31
	30	12.45	4.59	6.55	7.69	9.10	11.03
	50	12.41	3.62	5.72	8.34	9.28	11.45
	100	12.21	2.69	5.86	7.93	9.86	10.97
CEC2017	10	11.52	7.69	9.14	6.93	8.62	5.76
	30	14.90	2.55	5.31	7.79	10.28	12.62
	50	14.93	2.14	5.62	8.34	11.28	12.97
	100	12.66	1.86	5.76	7.76	10.48	13.21
CEC2022	10	14.83	2.25	5.08	7.92	10.50	13.17
	20	12.25	6.67	8.42	8.25	9.92	7.83
Average rank		13.05	3.96	6.56	7.91	9.75	10.83

Table 5. Ranking of algorithms with different RT when AS = 10Dim.

Test suite	Dim	ARO	RT=0.2Dim	RT=0.4Dim	RT=0.6Dim	RT=0.8Dim	RT=1.0Dim
CEC2014	10	12.38	4.59	5.41	5.48	7.79	8.48
	30	12.45	4.83	6.07	6.28	9.03	9.79
	50	12.41	4.66	5.21	7.55	9.45	11.41
	100	12.21	3.45	6.79	7.21	9.00	10.62
CEC2017	10	11.52	5.90	6.48	4.59	6.34	2.79
	30	14.90	2.66	4.86	7.48	10.07	12.34
	50	14.93	2.83	5.62	7.34	9.59	12.55
	100	12.66	3.24	5.93	8.45	11.45	12.83
CEC2022	10	14.83	2.00	4.83	7.00	9.25	10.83
	20	12.25	4.83	5.42	5.17	8.25	6.75
Average rank		13.05	3.90	5.66	6.65	9.02	9.84

Table 6. Ranking of algorithms with different RT when AS = 15Dim.

comprehensive comparison of IMARO and its variants, the experimental results are analyzed using the Friedman test and the Wilcoxon rank sum test.

The results of Friedman test for IMARO, IMARO variants and ARO are recorded in Table 10. Analysis of Table 10 shows that p value is not greater than 0.05 in all scenarios, which signifies that IMARO, as well as all IMARO variants, are significantly different from ARO. As can be seen in Fig. 5, IMARO is ranked first in the overall average ranking, which indicates that IMARO is the best in terms of overall performance, even though IMARO is inferior to IMARO-12 in terms of average ranking in CEC2017. detailed analysis is given below. In CEC2014, CEC2017 and CEC2022, all variants outperform ARO in terms of average ranking, which indicates that three

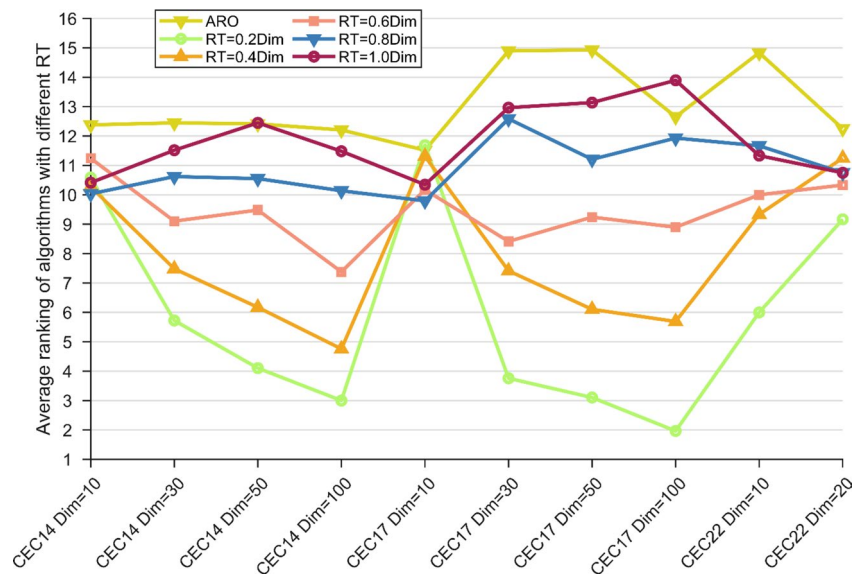


Figure 2. Ranking of algorithms with different RT when AS = 5Dim.

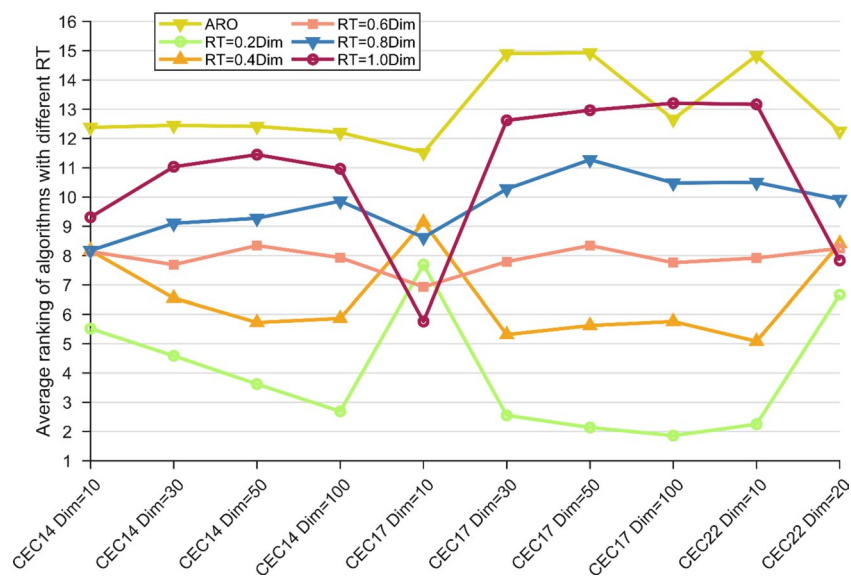


Figure 3. Ranking of algorithms with different RT when AS = 5Dim.

improvement strategies proposed in this paper have a positive impact on performance of ARO. The performance ranking of three IMARO variants combining one improvement strategy is IMARO-1 > IMARO-2 > IMARO-3. The performance ranking of IMARO variants combining two strategies is IMARO-12 > IMARO-13 > IMARO-23. combining the results of two rankings, it can be seen that the CRS strategy is an important part of the algorithm to improve the algorithm's performance by establishing the external archive, which preserves the historical information of excellent agents and improves performance of ARO by enhancing information exchange between agents. The INSS strategy ranks second in terms of its impact on algorithm performance. As can be seen from Table 10, IMARO-12 is improved compared to IMARO-1 and IMARO-13 is improved compared to IMARO-3, which indicates that the INSS strategy enhances the algorithm's ability to avoid local optima by applying perturbations to the optimal solution. In addition, IMARO-12, which is missing the RHS strategy, does not perform as well as IMARO, which illustrates that the RHS strategy improves performance of algorithm to some extent.

To further analyze the differences between ARO and each IMARO variant. Two-by-two comparisons of ARO and IMARO variants were performed using the Wilcoxon rank sum test. The results of test are shown in Table 11. In Table 11, the symbols “-”, “=”, and “+” indicate that IMARO variant is inferior, equal, and superior compared to ARO, respectively. It can be seen that three variants incorporating CRS strategy outperform basic ARO on most functions in different dimensions, indicating that CRS strategy is not restricted by dimensionality.

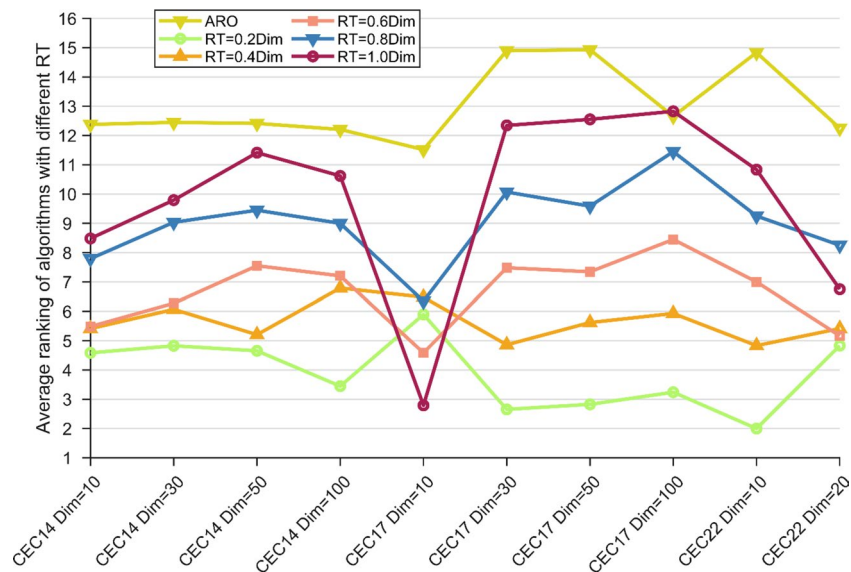


Figure 4. Ranking of algorithms with different RT when AS=5Dim.

Algorithm	A	B	C	D	Mathematical expression
ARO-RHS1	Yes	No	No	No	$X_i^{t+1} = \underbrace{X_i^t}_A + R \times (r_4 \times b_r - \underbrace{X_i^t}_B)$ $b_r = \underbrace{X_i^t}_C + \left(\frac{t_{\max} - t + 1}{t_{\max}} \right) \times r_4 \times g \times \underbrace{X_i^t}_D$
ARO-RHS2	No	Yes	No	No	
ARO-RHS3	No	No	Yes	No	
ARO-RHS4	No	No	No	Yes	
ARO-RHS5	No	Yes	Yes	No	
ARO-RHS6	No	No	Yes	Yes	
ARO-RHS7	Yes	No	Yes	No	
ARO-RHS8	No	Yes	No	Yes	
ARO-RHS9	Yes	No	No	Yes	
ARO-RHS10	Yes	Yes	No	No	

Table 7. The different random hiding strategies with RHS.

Algorithm	CEC2014 with Dim = 10/30/50/100				CEC2017 with Dim = 10/30/50/100				CEC2022 with Dim = 10/20		Average rank of CEC2014	Average rank of CEC2017	Average rank Of CEC2022	Average rank of CEC2014, CEC2017 and CEC2022
	10	30	50	100	10	30	50	100	10	20				
ARO	6.57	7.20	6.80	7.07	6.34	7.79	8.86	8.97	6.08	7.00	6.91	7.99	6.54	7.27
ARO-RHS1	6.20	5.40	5.93	6.37	6.38	6.17	6.55	7.52	7.83	5.58	5.98	6.66	6.71	6.39
ARO-RHS2	4.97	6.77	7.03	7.07	6.28	6.79	8.14	9.07	6.17	7.42	6.46	7.57	6.79	6.97
ARO-RHS3	6.23	6.00	5.07	5.33	7.14	6.28	5.14	4.90	6.25	7.00	5.66	5.86	6.63	5.93
ARO-RHS4	5.77	6.37	6.20	6.97	4.38	5.28	5.21	5.00	5.75	4.08	6.33	4.97	4.92	5.50
ARO-RHS5	7.17	5.03	5.90	5.83	7.24	6.24	5.03	4.17	6.08	7.92	5.98	5.67	7.00	6.06
ARO-RHS6	7.00	5.40	5.47	3.40	5.72	5.41	5.59	3.93	6.00	5.92	5.32	5.16	5.96	5.38
ARO-RHS7	6.27	4.53	4.57	3.53	6.41	5.03	3.14	2.10	4.75	5.67	4.73	4.17	5.21	4.60
ARO-RHS8	5.57	6.90	6.20	6.97	5.55	4.86	5.21	5.72	5.42	4.83	6.41	5.34	5.13	5.72
ARO-RHS9	6.00	7.53	7.00	7.60	5.76	5.83	5.83	7.52	5.58	5.08	7.03	6.23	5.33	6.37
ARO-RHS10	4.27	4.77	5.73	5.53	4.79	6.31	7.31	7.10	6.08	5.50	5.08	6.38	5.79	5.74

Table 8. Friedman scores of the ARO and its variations with RHS.

The IMARO variant incorporating INSS strategy does not improve much at $Dim = 10$ but performs better in higher dimensions. The RHS improves less than CRS and INSS, but also improves performance of ARO regardless of the dimensionality restriction. IMARO combining three strategies is inferior to ARO only seven times. Although IMARO outperforms ARO less often than IMARO-12 outperforms ARO. However, the IMARO solving function is inferior to ARO only 7 cases, much less than 18 cases of IMARO-12. In summary, IMARO is the best performer, and three improvement strategies proposed in this paper improve the ARO condition well.

Comparison with basic algorithms

The specific results of proposed IMARO and six basic algorithms in Table 3 for obtained results on the CEC 2014, CEC 2017 test suites ($Dim = 10, 30, 50, 100$), and CEC 2022 test suites ($Dim = 10, 20$), including the optimal value (Best), the mean (Ave), the standard deviation (Std), and the rankings (Rank), are presented in Tables S1–S10 in the Supplementary file.

In order to visualize the performance of each algorithm based on average value more visually. Figure 6 plots Mulberry diagram based on average rankings based on Rank data in Tables S1–S10. From Fig. 6, it can be generally concluded that IMARO significantly outperforms all competitors on all dimensions of the three test suites, which achieving the most significant victory with first overall ranking. In the next subsections, the experimental results will be precisely analyzed using statistical methods to avoid inaccurate conclusions due to statistical averages of experimental results.

Analysis of the Wilcoxon rank sum test results

The Wilcoxon rank sum test was used to examine whether IMARO and the other basic algorithms differed for each function. The Wilcoxon rank sum test was performed using the data in Tables S1–S10 in the Supplementary file with significance level $\alpha = 0.05$. The statistical results of the Wilcoxon rank sum test with significance level $\alpha = 0.05$ concerning the different dimensions of CEC2014, CEC2017, and CEC2022 are summarized in Table 12. The symbol “+” in Table 12 indicates that IMARO outperforms the competing algorithms. “–” indicates that IMARO is inferior to competitor. “=” indicates that there is no statistical difference between IMARO and comparison algorithm.

According to the statistics in Table 12, it can be learned that the number of “+” far exceeds the number of “–” in different dimensions of different test sets, which means that IMARO’s performance is significantly better than that of its competitors. The details are as follows. For CEC2014, IMARO is significantly superior to competing algorithms on at least 20 functions and is not weaker than ARO on at least 25 functions. At CEC2017, IMARO is clearly superior, with at least 25 functions better than comparison algorithm. On 10Dim, IMARO outperforms ARO on 21 functions, but IMARO outperforms ARO on both 50Dim and 100Dim. For CEC2022, IMARO

Strategy	IMARO-1	IMARO-2	IMARO-3	IMARO-12	IMARO-13	IMARO-23	IMARO
CRS	Yes	No	No	Yes	Yes	No	Yes
INSS	No	Yes	No	Yes	No	Yes	Yes
RHS	No	No	Yes	No	Yes	Yes	Yes

Table 9. Details of IMARO and six IMARO variants.

Algorithm	CEC2014 with Dim = 10/30/50/100				CEC2017 with Dim = 10/30/50/100				CEC2022 with Dim = 10/20		Average rank of CEC2014	Average rank of CEC2017	Average rank Of CEC2022	Average rank
	10 ^a	30 ^b	50 ^c	100 ^d	10 ^e	30 ^f	50 ^g	100 ^h	10 ⁱ	20 ^j				
ARO	6.50	6.57	6.83	6.77	5.90	7.48	7.76	7.86	6.17	7.42	7.25	6.67	6.79	6.77
IMARO-1	3.70	4.17	4.37	4.87	3.62	3.45	4.21	5.38	2.50	3.58	4.16	4.28	3.04	3.85
IMARO-2	5.00	4.27	3.73	2.90	5.48	5.34	4.93	3.93	5.25	5.33	4.92	3.98	5.29	4.52
IMARO-3	6.23	6.40	6.73	6.87	6.24	7.10	6.86	7.00	5.75	7.25	6.80	6.56	6.50	6.57
IMARO-12	2.20	3.30	2.90	2.90	2.72	2.21	1.69	1.59	3.83	1.92	2.05	2.83	2.88	2.76
IMARO-13	4.23	4.10	4.40	5.23	4.38	3.55	3.79	4.55	3.42	3.42	4.07	4.49	3.42	4.09
IMARO-23	5.33	4.67	4.70	4.07	4.69	5.03	4.69	3.72	5.83	4.92	4.53	4.69	5.38	4.90
IMARO	2.77	2.33	2.13	1.90	2.97	1.83	2.07	1.97	3.25	2.17	2.21	2.28	2.71	2.42

Table 10. The rankings of IMARO and its variants derived from the Friedman test. Best values are in [bold]. ^a p value is 1.58E–15, and the chi-square is 84.60. ^b p value is 6.25E–13, and the chi-square is 71.85. ^c p value is 6.53E–18, and the chi-square is 96.21. ^d p value is 2.41E–22, and the chi-square is 117.61. ^e p value is 2.03E–10, and the chi-square is 59.36. ^f p value is 3.65E–29, and the chi-square is 150.22. ^g p value is 6.81E–29, and the chi-square is 148.93. ^h p value is 2.64E–32, and the chi-square is 165.15. ⁱ p value is 2.47E–04, and the chi-square is 27.72. ^j p value is 5.64E–11, and the chi-square is 62.14.

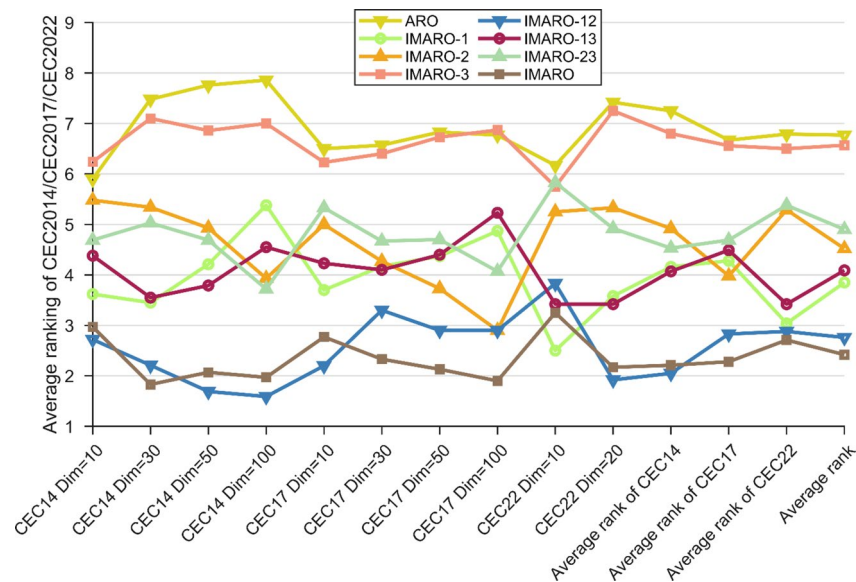


Figure 5. The ranking of IMARO and its six variants derived from the Friedman test.

Versus ARO + / = / -	CEC2014 with Dim = 10/30/50/100				CEC2017 with Dim = 10/30/50/100				CEC2022 with Dim = 10/20	
	10	30	50	100	10	30	50	100	10	20
IMARO-1	19/11/0	2019/9/2	2021/7/2	21/9/0	20/7/2	29/0/0	29/0/0	28/1/0	9/3/0	10/1/1
IMARO-2	7/23/0	18/12/0	20/10/0	25/4/1	4/25/0	21/8/0	23/6/0	28/1/0	0/12/0	6/6/0
IMARO-3	2/27/1	8/19/3	15/11/4	16/8/6	2/27/0	8/20/1	16/13/0	23/5/1	2/9/1	2/10/0
IMARO-12	28/1/1	22/4/4	26/1/3	25/0/5	21/6/2	28/0/1	29/0/0	29/0/0	10/0/2	12/0/0
IMARO-13	21/6/3	22/4/4	23/3/4	25/0/5	17/9/3	28/0/1	28/1/0	28/1/0	9/2/1	11/1/0
IMARO-23	8/20/2	21/5/4	20/6/4	24/1/5	6/23/0	21/7/1	24/5/0	26/2/1	1/10/1	7/5/0
IMARO	25/5/0	22/6/2	24/5/1	24/6/0	21/6/2	29/0/0	29/0/0	29/0/0	9/1/2	12/0/0

Table 11. Wilcoxon rank sum test results between the ARO and its variations.

outperforms comparison algorithm on at least 10 functions in all cases. Thus, the results of Wilcoxon rank sum test show that IMARO has significantly superior performance compared to six basic algorithms.

Analysis of the Friedman test results

In this section, the data in Tables S1–S10 are analyzed using Friedman’s test to further illustrate differences between IMARO and basic algorithms. Table 13 presents the results of Friedman test with significance level $\alpha=0.05$ for all seven algorithms for all dimensions using the CEC2014, CEC2017 and CEC2022 test suites. As can be seen from Table 13, the Friedman p value for all dimensions is less than 0.05, which indicates that performance of seven algorithms involved in the experiments is significantly different in all cases. In addition, Fig. 7 shows a ranking graph based on test results in Table 13 to visualize Friedman test results of each algorithm.

Based on the results in Fig. 7, it can be seen that IMARO outperforms all compared algorithms in all cases. The average performance of IMARO in the three test suites is ranked as $\text{CEC2017} > \text{CEC2022} > \text{CEC2014}$. The details are as follows: IMARO has a better performance in lower dimension in CEC2014, and its performance in higher dimension decreases but still remains the best algorithm in general. For CEC2017 and CEC2022, IMARO’s performance is not limited by dimension change, and it still performs best in high dimensions.

Analysis of the convergence speed and accuracy

In this section, we analyze the convergence ability of IMARO by plotting convergence curve, based on experimental data. Figure 8 shows the average convergence curves over 30 runs for IMARO and six basic algorithms. For conciseness, three functions from each of three test suites (CEC2014/CEC2017 Dim = 30, CEC2022 Dim = 20) are selected for presentation in this section, and average convergence curves for all functions in all dimensions are shown in Figs. S1–S10 in the Supplementary file. As can be seen in Fig. 8, IMARO has the best convergence accuracy and convergence speed on given few functions, except for F18 and F29 of CEC2017. IMARO can converge to a better solution on these two functions, although the convergence speed is slower than that of the other competitors in the early stage. Thus, IMARO performs best on all three test suites with regard to convergence performance.

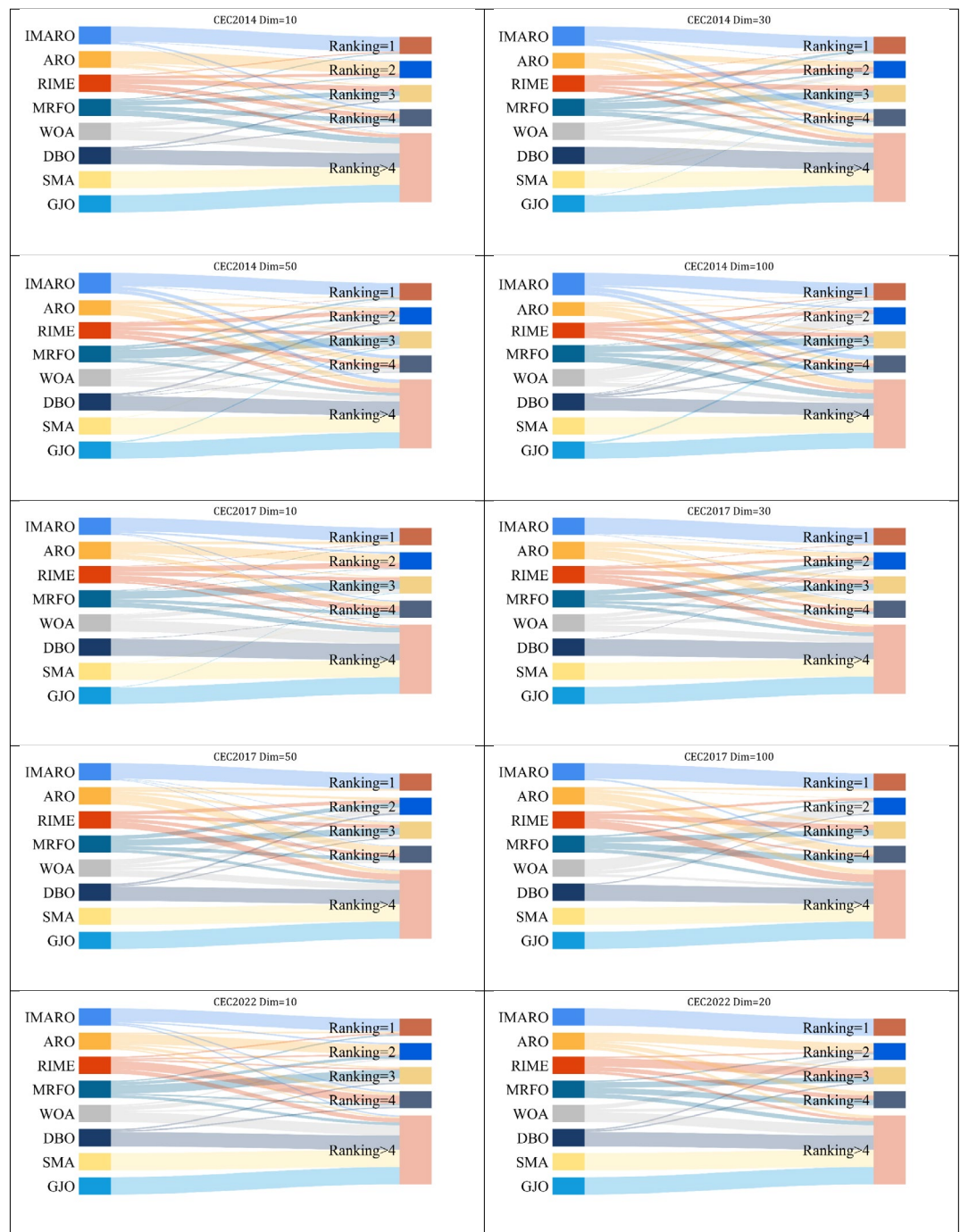


Figure 6. The ranking stack chart of IMARO and six basic algorithms.

Comparison with improved algorithms

The specific results of proposed IMARO and seven improved algorithms in Table 3 for the obtained results on the CEC 2014, CEC 2017 test suites ($Dim = 10, 30, 50, 100$), and CEC 2022 test suites ($Dim = 10, 20$), including the optimal value (Best), the mean (Ave), the standard deviation (Std), and the rankings (Rank), are presented in Tables S11–S20 in the Supplementary file. To visualize more the performance of each algorithm based on mean index. Figure 9 shows a bar stacking graph of mean-based rankings based on Rank's data in Tables S11–S20. We categorized rankings into six major categories: average rank 1st, average rank 2nd, average rank 3rd, average rank 4th, and other rankings. In Fig. 8, it can be roughly concluded that IMARO significantly outperforms all competitors on all dimensions of three test suites. In the next subsections, statistical methods are used to accurately analyze the experimental results in order to avoid inaccurate conclusions due to statistical averages of experimental results.

+/-/-IMARO versus	CEC2014 with Dim = 10/30/50/100				CEC2017 with Dim = 10/30/50/100				CEC2022 with Dim = 10/20	
	10	30	50	100	10	30	50	100	10	20
ARO	28/1/1	22/4/4	26/1/3	25/0/5	21/6/2	28/0/1	29/0/0	29/0/0	10/0/2	12/0/0
RIME	29/1/0	28/0/2	30/0/0	29/0/1	28/1/0	28/1/0	28/1/0	29/0/0	11/1/0	11/1/0
MRFO	26/1/3	21/3/6	21/3/6	20/2/8	25/4/0	27/2/0	28/1/0	25/1/3	10/1/1	11/1/0
WOA	29/1/0	30/0/0	28/1/1	27/0/3	29/0/0	29/0/0	29/0/0	29/0/0	12/0/0	12/0/0
DBO	29/1/0	29/0/1	27/2/1	28/1/1	28/1/0	29/0/0	29/0/0	29/0/0	12/0/0	12/0/0
SMA	27/0/3	24/0/6	22/3/5	23/1/6	28/1/0	28/1/0	28/1/0	29/0/0	10/1/1	11/1/0
GJO	30/0/0	29/0/1	29/1/0	28/1/1	29/0/0	29/0/0	29/0/0	29/0/0	12/0/0	12/0/0

Table 12. Wilcoxon rank sum test results between the IMARO and basic algorithms.

Algorithm	CEC2014 with Dim = 10/30/50/100				CEC2017 with Dim = 10/30/50/100				CEC2022 with Dim = 10/20		Average rank of CEC2014	Average rank of CEC2017	Average rank of CEC2022	Average rank
	10 ^a	30 ^b	50 ^c	100 ^d	10 ^e	30 ^f	50 ^g	100 ^h	10 ⁱ	20 ^j				
IMARO	1.33	1.73	1.67	1.83	1.31	1.10	1.10	1.10	1.50	1.00	1.64	1.16	1.25	1.35
ARO	2.37	3.33	3.98	4.10	2.38	3.45	4.03	4.59	2.58	3.33	3.45	3.61	2.96	3.34
RIME	4.03	4.07	4.10	4.30	4.03	3.59	3.72	3.55	4.17	3.50	4.13	3.72	3.83	3.89
MRFO	3.38	2.93	2.93	2.92	3.21	3.10	3.00	2.76	3.17	3.42	3.04	3.02	3.29	3.12
WOA	7.27	7.50	7.12	6.97	7.69	7.83	7.66	7.62	7.83	7.67	7.21	7.70	7.75	7.55
DBO	5.97	6.03	6.07	6.53	5.66	6.31	6.34	6.45	5.83	6.17	6.15	6.19	6.00	6.11
SMA	4.78	3.50	3.20	2.68	4.97	3.93	3.41	3.17	3.92	4.33	3.54	3.87	4.13	3.85
GJO	6.87	6.90	6.93	6.67	6.76	6.69	6.72	6.76	7.00	6.58	6.84	6.73	6.79	6.79
<i>p</i> value	9.70E-31	5.98E-29	5.52E-27	3.14E-27	1.65E-31	4.80E-33	3.18E-32	4.84E-34	3.60E-12	9.84E-12	N/A	N/A	N/A	N/A

Table 13. The rankings of IMARO and basic algorithms derived from the Friedman test. Best values are in [bold].

Analysis of the Wilcoxon rank sum test results

The Wilcoxon rank test was conducted using data from Supplementary file Tables S11–S20 with the significance level $\alpha = 0.05$. The results of Wilcoxon rank sum test for the CEC2014, CEC2017 and CEC2022 test suites are summarized in Table 14. The details are given below.

For the CEC2014 test suite, IMARO is superior (inferior) to AFDBARO on 95(13) functions, FDBARO on 99(12) functions, DTSMA on 102(15) functions, RCLSMOAO on 93(20) functions, IHAOAVOA on 87(30)

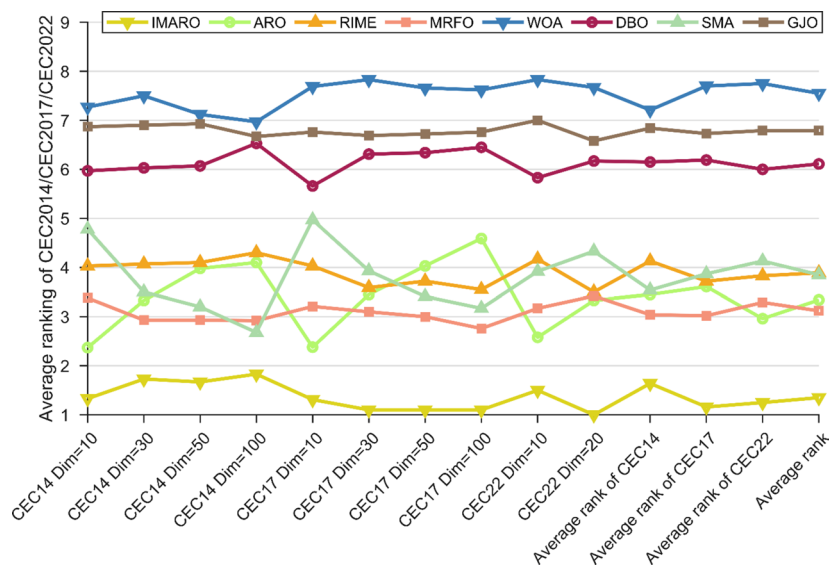


Figure 7. The ranking of IMARO and six basic algorithms derived from the Friedman test.

functions, MSCSO on 92(25) functions, and RLTLBO on 88(27) functions. That is to say IMARO is superior to other improved competitors on the CEC2014 test suite.

For the CEC2017 test suite, IMARO is superior (inferior) to AFDBARO on 108(2) functions, FDBARO on 99(3) functions, DTSMA on 108(0) functions, RCLSMOAO on 112(0) functions, IHAOAVOA on 115(0) functions, MSCSO on 116(0) functions, and RLTLBO on 101(7) functions. That is to say IMARO is superior to other improved competitors on the CEC2017 test suite.

For the CEC2022 test suite, IMARO is superior (inferior) to AFDBARO on 21(1) functions, FDBARO on 19(2) functions, DTSMA on 20(2) functions, RCLSMOAO on 23(0) functions, IHAOAVOA on 23(0) functions, MSCSO on 23(0) functions, and RLTLBO on 20(3) functions. That is to say IMARO is superior to other improved competitors on the CEC2022 test suite.

The Wilcoxon rank sum test supports the conclusion that IMARO outperforms the seven improved algorithms on CEC2014, CEC2017 and CEC2022. The advantage of IMARO reaches more than 72.5% on CEC2014, more than 85.4% on CEC2017, and more than 79.1% on CEC2022.

Analysis of the Friedman test results

In this section, the data in Tables S11–S20 will be analyzed using the Friedman test to further illustrate the differences between IMARO and improved competitors. The Friedman test results for all eight algorithms performed using CEC2014, CEC2017, and CEC2022 test suites for all dimensions are shown in Table 15, with a significance level of $\alpha = 0.05$. From Table 15, it can be seen that the Friedman p value for all cases is less than 0.05, which indicates that there is a significant difference between IMARO and competitors. In addition, Fig. 10 shows a ranking plot based on the test results in Table 15 to visualize the Friedman test results for each algorithm.

Based on the results in Fig. 10, it can be seen that IMARO outperforms all improved competitors in all cases. IMARO's average performance in three test suites is ranked as CEC2017 > CEC2022 > CEC2014. Details are as follows: the low-dimensional performance of IMARO is better in CEC2014, and its high-dimensional performance decreases, but it is still the best algorithm overall. For CEC2017 and CEC2022, IMARO's performance is not limited by the change of dimension, and the gap between its performance and that of competitors becomes bigger and bigger as the dimension increases.

Mechanical engineering problems

In this section, in order to verify the scalability of the algorithm, a feasibility study was conducted on six mechanical engineering problems in practical constraint optimization. The specific details are shown in Table 16, where D represents the dimension of the optimization problem, g is the number of inequality constraints, h indicates the number of equality constraints. The specific optimization comparison results are shown in Table 17. The

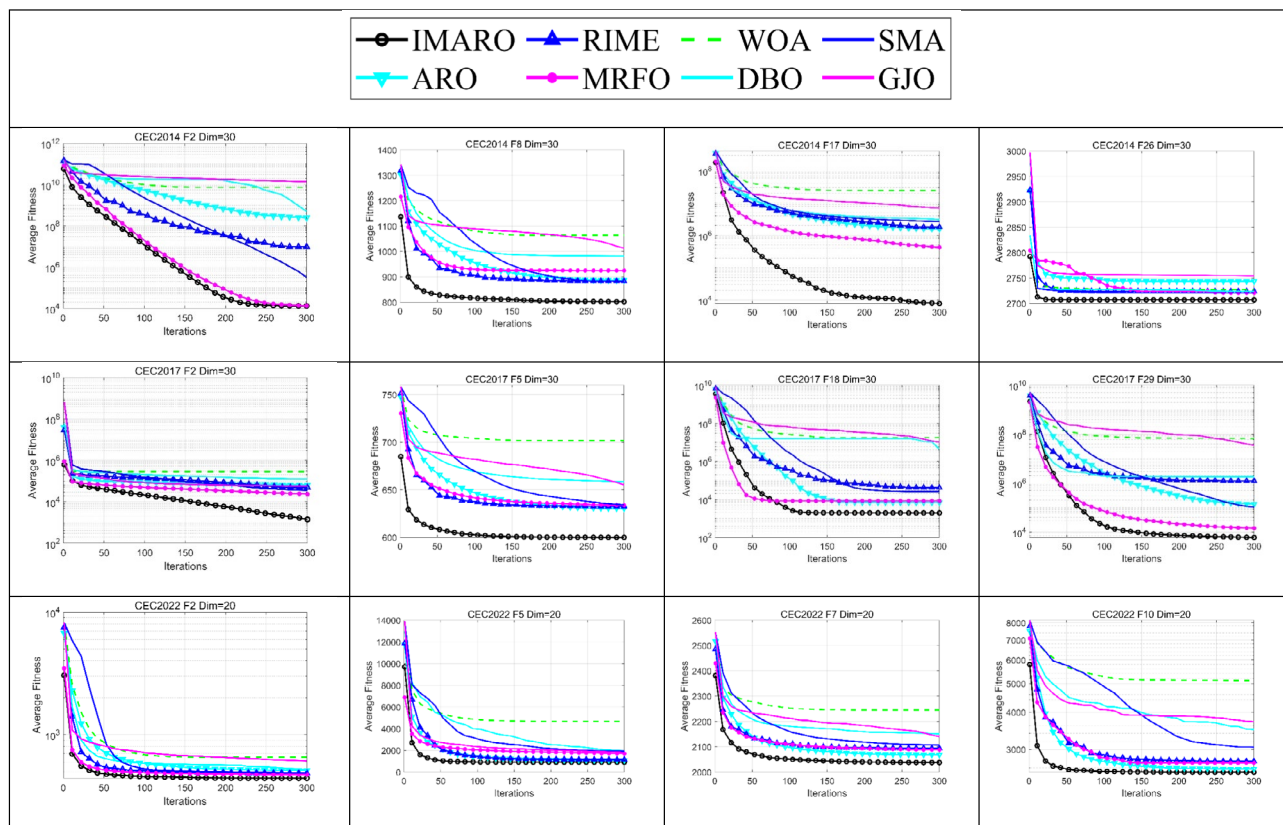


Figure 8. The convergence curves of IMARO and six basic algorithms.

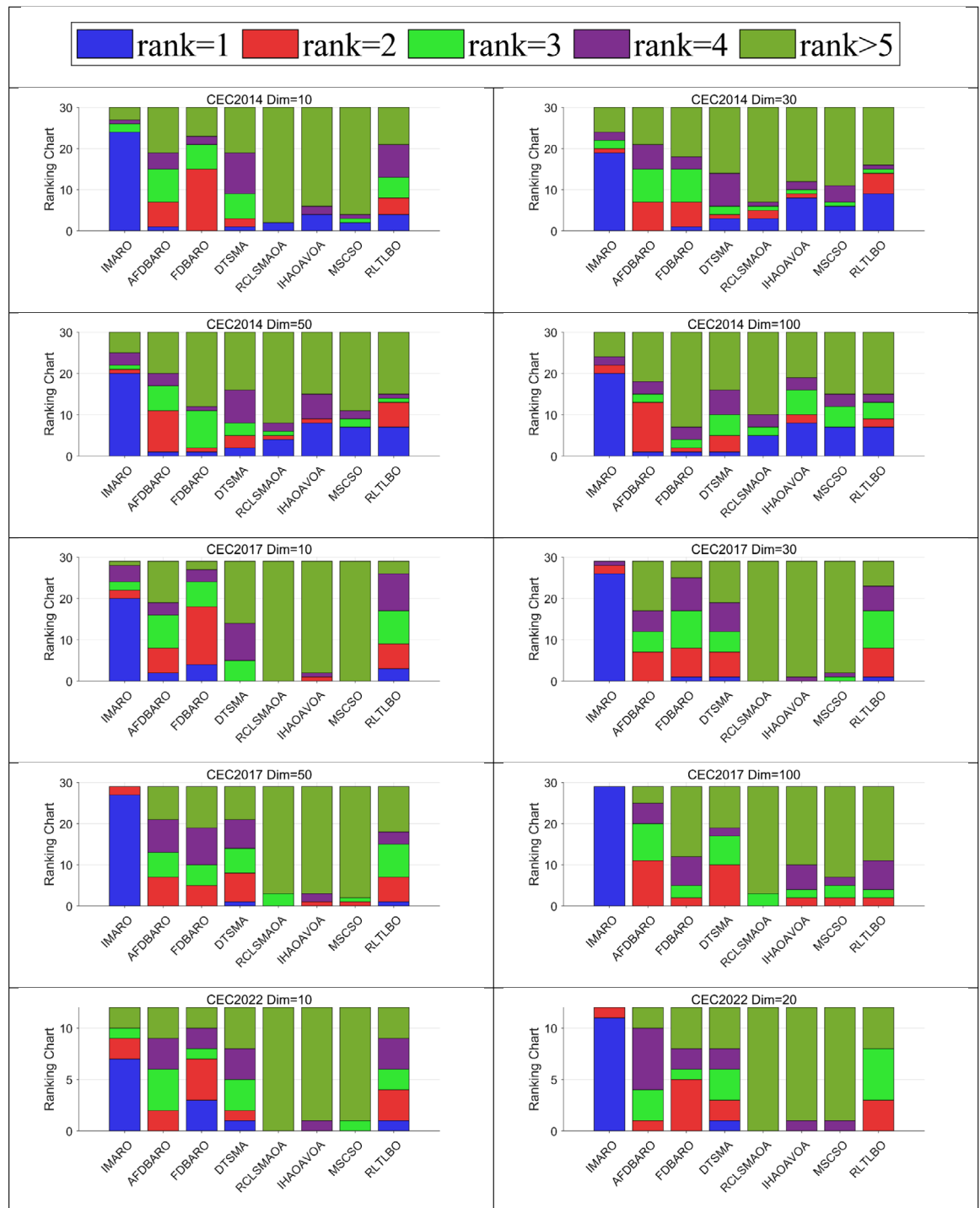


Figure 9. The ranking stack chart of IMARO and seven improved algorithms.

results indicate that compared to other competitors, IMARO performs the best on six mechanical engineering problems, which demonstrates the feasibility of IMARO for practical optimization problems.

Conclusions

An improved multi-strategy artificial rabbits optimization is proposed to address the drawbacks of slow convergence speed of optimization and easy to fall into local traps. In IMARO, a roulette fitness distance balanced avoidance strategy is proposed so that the rabbits can find better avoidance positions in a more reasonable way. Meanwhile, to improve the defect that ARO is easy to fall into local optimum, an improved non-monopoly search strategy based on Gaussian and Cauchy operators is designed to improve algorithm's ability to achieve the global optimal solution. Finally, a covariance restart strategy is designed to improve the population diversity when a stagnant exploitation occurs, and to improve the convergence accuracy and convergence speed of ARO.

+ / = / - IMARO versus	CEC2014 with <i>Dim</i> = 10/30/50/100					CEC2017 with <i>Dim</i> = 10/30/50/100					CEC2022 with <i>Dim</i> = 10/20		
	10	30	50	100	Total	10	30	50	100	Total	10	20	Total
AFDBARO	26/3/1	21/5/4	26/1/3	22/3/5	95/12/13	23/5/1	28/0/1	29/0/0	28/1/0	108/6/2	9/2/1	12/0/0	21/2/1
FDBARO	25/5/0	24/2/4	26/1/3	24/1/5	99/9/12	18/9/2	24/4/1	28/1/0	29/0/0	99/14/3	7/3/2	12/0/0	19/3/2
DTSMA	26/0/4	25/0/5	25/3/2	26/0/4	102/3/15	27/2/0	25/4/0	27/2/0	29/0/0	108/8/0	9/1/2	11/1/0	20/2/2
RCLSMOAO	27/0/3	22/2/6	21/4/5	23/1/6	93/7/20	29/0/0	28/1/0	27/2/0	28/1/0	112/4/0	11/1/0	12/0/0	23/1/0
IHAOAVOA	25/0/5	21/1/8	20/2/8	21/0/9	87/3/30	28/1/0	29/0/0	29/0/0	29/0/0	115/1/0	11/1/0	12/0/0	23/1/0
MSCSO	26/1/3	22/1/7	22/1/7	22/0/8	92/3/25	29/0/0	29/0/0	29/0/0	29/0/0	116/0/0	11/1/0	12/0/0	23/1/0
RLTLBO	24/1/5	21/2/7	21/2/7	22/0/8	88/5/27	19/5/5	26/2/1	27/1/1	29/0/0	101/8/7	8/1/3	12/0/0	20/1/3

Table 14. Wilcoxon rank sum test results between the IMARO and improved algorithms. Best values are in [bold].

Algorithm	CEC2014 with <i>Dim</i> = 10/30/50/100				CEC2017 with <i>Dim</i> = 10/30/50/100				CEC2022 with <i>Dim</i> = 10/20		Average rank of CEC2014	Average rank of CEC2017	Average rank of CEC2022	Average rank
	10	30	50	100	10	30	50	100	10	20				
IMARO	1.87	2.60	2.40	2.53	1.79	1.17	1.07	1.00	2.17	1.08	2.35	1.26	1.63	1.74
AFDBARO	3.73	3.90	3.67	3.80	3.45	3.90	3.66	3.14	3.67	3.83	3.78	3.53	3.75	3.69
FDBARO	3.30	4.02	4.70	5.58	2.52	3.28	3.97	5.00	2.67	3.58	4.40	3.69	3.13	3.74
DTSMA	4.23	4.72	4.77	4.87	4.69	3.83	3.72	3.83	3.83	3.50	4.65	4.02	3.67	4.11
RCLS-MAOA	7.35	6.50	6.27	5.83	7.55	7.48	7.07	6.76	7.58	7.50	6.49	7.22	7.54	7.08
IHAOAVOA	6.00	5.02	4.63	3.98	6.62	6.52	6.24	5.45	6.58	6.42	4.91	6.21	6.50	5.87
MSCSO	5.58	5.23	5.23	4.85	6.24	6.34	6.41	5.62	6.08	6.67	5.23	6.16	6.38	5.92
RLTLBO	3.93	4.02	4.33	4.55	3.14	3.48	3.86	5.21	3.42	3.42	4.21	3.92	3.42	3.85
<i>p</i> value	6.53E-20	3.68E-08	9.09E-08	1.01E-06	3.49E-29	7.16E-29	4.15E-25	1.44E-20	1.53E-09	1.50E-11	N/A	N/A	N/A	N/A

Table 15. The rankings of IMARO and improved algorithms derived from the Friedman test. Best values are in [bold].

The excellent performance of IMARO is verified by comparing it with six basic algorithms and seven improved algorithms in CEC2014, CEC2017 and CEC2022. Using Wilcoxon rank sum test and Friedman test, we confirm the significant superiority of IMARO over its competitors with results can be found in “[Experimental results and discussion](#)” section. Moreover, six engineering constrained optimization problems further confirm the potential of IMARO to solve real-world problems.

In future work, we will continue to explore the application of IMARO in other areas such as image segmentation, wireless sensor coverage and UAV mission planning, which are challenging practical problems. In addition, as real-world optimization problems become more complex, multi-objective and constrained optimization problems become more important. We will aim to focus on creating a multi-objective version of IMARO to provide more comprehensive and high-quality solutions to optimization problems.

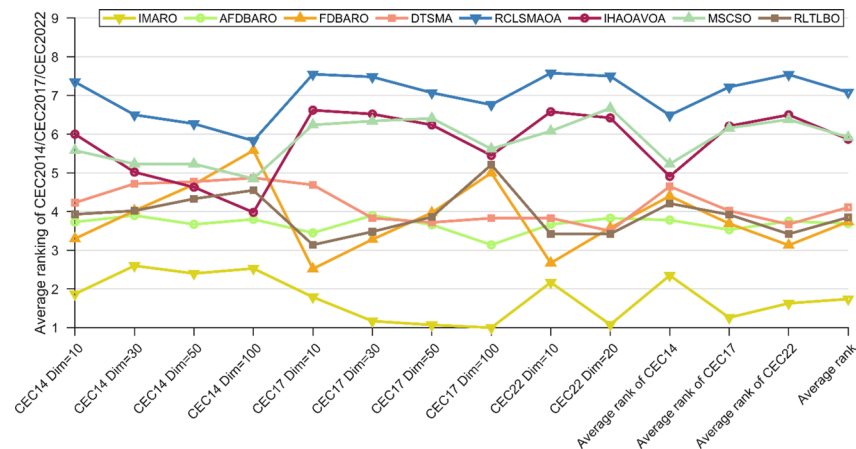


Figure 10. The ranking of IMARO and seven improved algorithms derived from the Friedman test.

No	Problem name	D	g	h
F1	Tension/compression spring design	3	3	0
F2	Pressure vessel design	4	4	0
F3	Three-bar truss design problem	2	3	0
F4	Welded beam design	4	5	0
F5	Speed reducer design problem	7	11	0
F6	Gear train design problem	4	1	1

Table 16. Details of mechanical engineering problem.

No	Index	IMARO	ARO	RIME	MRFO	SMA	AFDBARO	FDBARO	DTSMA	IHAAVOA	RLTLBO
F1	Best	0.012667	0.012724	0.013039	0.012667	0.012669	0.012665	0.012710	0.012670	0.012701	0.012709
	Mean	0.012669	0.013810	0.014029	0.012864	0.012705	0.012675	0.013129	0.012711	0.012722	0.013400
	Std	2.860E−06	9.277E−04	1.271E−03	1.945E−04	2.621E−05	1.208E−05	3.591E−04	2.387E−05	2.901E−05	9.172E−04
	Rank	1	9	10	6	3	2	7	4	5	8
F2	Best	5885.3328	5910.9544	6097.6383	5980.9458	5886.8646	5885.7288	6100.4998	5981.3226	5885.3328	5999.6221
	Mean	5885.7566	6590.5727	6571.5572	6289.8275	5957.3206	5886.2301	6577.0760	6345.8081	5948.0787	6430.0756
	Std	9.371E−01	4.748E+02	4.360E+02	2.478E+02	8.245E+01	3.794E−01	3.725E+02	4.190E+02	1.008E+02	2.769E+02
	Rank	1	10	8	5	4	2	9	6	3	7
F3	Best	263.8958	263.8963	263.8981	263.8958	263.8959	263.8958	263.8958	263.8958	263.8958	263.8959
	Mean	263.8958	263.9055	263.9539	263.8960	263.8959	263.8958	263.8958	263.8959	263.8958	263.9000
	Std	1.145E−06	1.246E−02	5.412E−02	2.195E−04	6.813E−05	6.267E−06	7.700E−06	1.878E−05	5.411E−06	4.647E−03
	Rank	1	9	10	7	6	3	4	5	2	8
F4	Best	1.692768	1.694175	1.747905	1.692768	1.692777	1.692768	1.692768	1.692788	1.692768	1.698074
	Mean	1.692768	1.711287	1.805953	1.700194	1.692826	1.692768	1.706501	1.692883	1.692769	1.717754
	Std	7.584E−16	3.487E−02	6.036E−02	2.322E−02	4.617E−05	2.239E−14	3.223E−02	1.346E−04	5.579E−07	2.100E−02
	Rank	1	8	10	6	4	2	7	5	3	9
F5	Best	2994.4711	2994.6940	3003.7067	2994.4711	2994.5097	2994.4711	2994.4711	2994.5145	2994.4712	2996.6232
	Mean	2994.4711	2995.6855	3037.5256	2994.4711	2994.5487	2994.4711	2996.3492	2994.7180	2994.4719	3004.1789
	Std	0.000E+00	1.187E+00	1.918E+01	8.440E−13	3.042E−02	7.044E−06	3.261E+00	1.965E−01	9.946E−04	4.289E+00
	Rank	1	7	10	2	5	3	8	6	4	9
F6	Best	2.701E−12	2.308E−11	2.701E−12	2.701E−12	2.701E−12	2.701E−12	2.701E−12	2.701E−12	2.308E−11	2.701E−12
	Mean	8.814E−12	1.173E−10	3.235E−10	3.474E−10	1.696E−11	4.486E−10	4.761E−10	2.026E−10	1.038E−09	9.268E−10
	Std	9.843E−12	2.721E−10	4.545E−10	4.484E−10	9.843E−12	6.317E−10	7.687E−10	4.162E−10	5.727E−10	9.121E−10
	Rank	1	3	5	6	2	7	8	4	10	9

Table 17. Results of mechanical engineering problem.

Data availability

The data is provided within the manuscript and supplementary files.

Received: 30 May 2024; Accepted: 30 July 2024

Published online: 07 August 2024

References

- Jin, B., Cruz, L. & Gonçalves, N. Pseudo RGB-D face recognition. *IEEE Sens. J.* <https://doi.org/10.1109/JSEN.2022.3197235> (2022).
- Jin, B. & Vai, M. I. An adaptive ultrasonic backscattered signal processing technique for accurate object localization based on the instantaneous energy density level. *J. Med. Imaging Heal. Inform.* <https://doi.org/10.1166/jmihi.2015.1493> (2015).
- Wu, G., Pedrycz, W., Suganthan, P. N. & Mallipeddi, R. A variable reduction strategy for evolutionary algorithms handling equality constraints. *Appl. Soft Comput. J.* <https://doi.org/10.1016/j.asoc.2015.09.007> (2015).
- Das, S., Mullick, S. S. & Suganthan, P. N. Recent advances in differential evolution—an updated survey. *Swarm Evol. Comput.* <https://doi.org/10.1016/j.swevo.2016.01.004> (2016).
- Mamoudan, M. M., Jafari, A., Mohammadnazari, Z., Nasiri, M. M. & Yazdani, M. Hybrid machine learning-metaheuristic model for sustainable agri-food production and supply chain planning under water scarcity. *Resour. Environ. Sustain.* <https://doi.org/10.1016/j.resenv.2023.100133> (2023).
- Khalili-Fard, A. *et al.* Multi-objective optimization of closed-loop supply chains to achieve sustainable development goals in uncertain environments. *Eng. Appl. Artif. Intell.* <https://doi.org/10.1016/j.engappai.2024.108052> (2024).
- Linganathan, S. & Singamsetty, P. Genetic algorithm to the bi-objective multiple travelling salesman problem. *Alex. Eng. J.* <https://doi.org/10.1016/j.aej.2024.01.048> (2024).
- Li, X., Zhang, S. & Shao, P. Discrete artificial bee colony algorithm with fixed neighborhood search for traveling salesman problem. *Eng. Appl. Artif. Intell.* <https://doi.org/10.1016/j.engappai.2023.107816> (2024).
- Gupta, S. & Deep, K. Improved sine cosine algorithm with crossover scheme for global optimization. *Knowl. Based Syst.* <https://doi.org/10.1016/j.knsys.2018.12.008> (2019).
- Liu, Q., Li, N., Jia, H., Qi, Q. & Abualigah, L. A chimp-inspired remora optimization algorithm for multilevel thresholding image segmentation using cross entropy. *Artif. Intell. Rev.* <https://doi.org/10.1007/s10462-023-10498-0> (2023).
- Zhao, Q., Li, C., Zhu, D. & Xie, C. Coverage optimization of wireless sensor networks using combinations of PSO and chaos optimization. *Electron.* <https://doi.org/10.3390/electronics11060853> (2022).
- Ou, Y. *et al.* An improved grey wolf optimizer with multi-strategies coverage in wireless sensor networks. *Symmetry (Basel)*. <https://doi.org/10.3390/sym16030286> (2024).
- Li, Y., Han, T., Zhou, H., Tang, S. & Zhao, H. A novel adaptive L-SHADE algorithm and its application in UAV swarm resource configuration problem. *Inf. Sci. (NY)*. <https://doi.org/10.1016/j.ins.2022.05.058> (2022).
- Tang, A. D., Han, T., Zhou, H. & Xie, L. An improved equilibrium optimizer with application in unmanned aerial vehicle path planning. *Sensors*. <https://doi.org/10.3390/s21051814> (2021).
- Holland, J. H. *Adaptation in Natural and Artificial Systems* (University of Michigan Press, 1992).
- Sarker, R. A., Elsayed, S. M. & Ray, T. Differential evolution with dynamic parameters selection for optimization problems. *IEEE Trans. Evol. Comput.* <https://doi.org/10.1109/TEVC.2013.2281528> (2014).
- Fogel, D. B. Applying evolutionary programming to selected traveling salesman problems. *Cybern. Syst.* <https://doi.org/10.1080/01969729308961697> (1993).
- Beyer, H.-G. & Schwefel, H.-P. Evolution strategies—A comprehensive introduction. *Nat. Comput.* <https://doi.org/10.1023/A:1015059928466> (2002).
- Yang, X. *Nature-Inspired Metaheuristic Algorithms* (Luniver Press, 2010).
- Rashedi, E., Nezamabadi-pour, H. & Saryazdi, S. GSA: A gravitational search algorithm. *Inf. Sci. (NY)*. <https://doi.org/10.1016/j.ins.2009.03.004> (2009).
- Mirjalili, S. SCA: A sine cosine algorithm for solving optimization problems. *Knowl. Based Syst.* <https://doi.org/10.1016/j.knsys.2015.12.022> (2016).
- Mirjalili, S., Mirjalili, S. M. & Hatamlou, A. Multi-verse optimizer: A nature-inspired algorithm for global optimization. *Neural Comput. Appl.* <https://doi.org/10.1007/s00521-015-1870-7> (2016).
- Hashim, F. A., Houssein, E. H., Mabrouk, M. S., Al-Atabany, W. & Mirjalili, S. Henry gas solubility optimization: A novel physics-based algorithm. *Futur. Gener. Comput. Syst.* <https://doi.org/10.1016/j.future.2019.07.015> (2019).
- Deng, L. & Liu, S. Snow ablation optimizer: A novel metaheuristic technique for numerical optimization and engineering design. *Expert Syst. Appl.* <https://doi.org/10.1016/j.eswa.2023.120069> (2023).
- Hashim, F. A., Hussain, K., Houssein, E. H., Mabrouk, M. S. & Al-Atabany, W. Archimedes optimization algorithm: A new metaheuristic algorithm for solving optimization problems. *Appl. Intell.* <https://doi.org/10.1007/s10489-020-01893-z> (2021).
- Rao, R. V., Savsani, V. J. & Vakharia, D. P. Teaching-learning-based optimization: A novel method for constrained mechanical design optimization problems. *CAD Comput. Aided Des.* <https://doi.org/10.1016/j.cad.2010.12.015> (2011).
- Bayzidi, H., Talatahari, S., Saraee, M. & Lamarche, C. P. Social network search for solving engineering optimization problems. *Comput. Intell. Neurosci.* <https://doi.org/10.1155/2021/8548639> (2021).
- Zhang, Y. & Jin, Z. Group teaching optimization algorithm: A novel metaheuristic method for solving global optimization problems. *Expert Syst. Appl.* <https://doi.org/10.1016/j.eswa.2020.113246> (2020).
- Sağ, T. PVS: A new population-based vortex search algorithm with boosted exploration capability using polynomial mutation. *Neural Comput. Appl.* <https://doi.org/10.1007/s00521-022-07671-x> (2022).
- Ma, B., Hu, Y., Lu, P. & Liu, Y. Running city game optimizer: A game-based metaheuristic optimization algorithm for global optimization. *J. Comput. Des. Eng.* <https://doi.org/10.1093/jcde/qwac131> (2023).
- Kumar, M., Kulkarni, A. J. & Satapathy, S. C. Socio evolution & learning optimization algorithm: A socio-inspired optimization methodology. *Futur. Gener. Comput. Syst.* <https://doi.org/10.1016/j.future.2017.10.052> (2018).
- Kennedy, J. & Eberhart, R.: Particle swarm optimization. In *IEEE International Conference on Neural Networks—Conference Proceedings* (1995).
- Dorigo, M. & Di Caro, G.: Ant colony optimization: A new meta-heuristic. In *Proceedings of the 1999 Congress on Evolutionary Computation, CEC 1999* (1999).
- Mirjalili, S. & Lewis, A. The whale optimization algorithm. *Adv. Eng. Softw.* <https://doi.org/10.1016/j.advengsoft.2016.01.008> (2016).
- Mirjalili, S., Mirjalili, S. M. & Lewis, A. Grey wolf optimizer. *Adv. Eng. Softw.* <https://doi.org/10.1016/j.advengsoft.2013.12.007> (2014).
- Jia, H., Peng, X. & Lang, C. Remora optimization algorithm. *Expert Syst. Appl.* <https://doi.org/10.1016/j.eswa.2021.115665> (2021).
- Abualigah, L., Elaziz, M. A., Sumari, P., Geem, Z. W. & Gandomi, A. H. Reptile search algorithm (RSA): A nature-inspired metaheuristic optimizer. *Expert Syst. Appl.* <https://doi.org/10.1016/j.eswa.2021.116158> (2022).
- Xie, L. *et al.* Tuna swarm optimization: A novel swarm-based metaheuristic algorithm for global optimization. *Comput. Intell. Neurosci.* <https://doi.org/10.1155/2021/9210050> (2021).

39. Seyyedabbasi, A. & Kiani, F. Sand cat swarm optimization: A nature-inspired algorithm to solve global optimization problems. *Eng. Comput.* <https://doi.org/10.1007/s00366-022-01604-x> (2023).
40. Wang, L., Cao, Q., Zhang, Z., Mirjalili, S. & Zhao, W. Artificial rabbits optimization: A new bio-inspired meta-heuristic algorithm for solving engineering optimization problems. *Eng. Appl. Artif. Intell.* <https://doi.org/10.1016/j.engappai.2022.105082> (2022).
41. Bakır, H. Dynamic fitness-distance balance-based artificial rabbits optimization algorithm to solve optimal power flow problem. *Expert Syst. Appl.* <https://doi.org/10.1016/j.eswa.2023.122460> (2024).
42. Alamir, N., Kamel, S., Hassan, M. H. & Abdelkader, S. M. An effective quantum artificial rabbits optimizer for energy management in microgrid considering demand response. *Soft Comput.* <https://doi.org/10.1007/s00500-023-08814-5> (2023).
43. Bennet, G. S. D. & Subramaniam Nachimuthu, D. Solar PV system with modified artificial rabbit optimization algorithm for MPPT. *Electr. Eng.* <https://doi.org/10.1007/s00202-023-02231-5> (2024).
44. Wolpert, D. H. & Macready, W. G. No free lunch theorems for optimization. *IEEE Trans. Evol. Comput.* <https://doi.org/10.1109/4235.585893> (1997).
45. Cao, Q. et al. Vibration state identification of hydraulic units based on improved artificial rabbits optimization algorithm. *Biomimetics*. <https://doi.org/10.3390/biomimetics8020243> (2023).
46. Abd Elaziz, M., Dahou, A., Mabrouk, A., El-Sappagh, S. & Aseeri, A. O. An efficient artificial rabbits optimization based on mutation strategy for skin cancer prediction. *Comput. Biol. Med.* <https://doi.org/10.1016/j.compbiomed.2023.107154> (2023).
47. SeyedOskoue, S. L., Sojoudizadeh, R., Milanchian, R. & Azizian, H. Shape and size optimization of truss structure by means of improved artificial rabbits optimization algorithm. *Eng. Optim.* <https://doi.org/10.1080/0305215X.2024.2308577> (2024).
48. Hu, G., Jing, W. & Houssein, E. H. Elite-based feedback boosted artificial rabbits-inspired optimizer with mutation and adaptive group: A case study of degree reduction for ball NURBS curves. *Soft Comput.* <https://doi.org/10.1007/s00500-023-09023-w> (2023).
49. Wang, Y., Huang, L., Zhong, J. & Hu, G. LARO: Opposition-based learning boosted artificial rabbits-inspired optimization algorithm with lévy flight. *Symmetry (Basel)*. <https://doi.org/10.3390/sym14112282> (2022).
50. Liang, J. J., Qu, B. Y. & Suganthan, P. N.: Problem definitions and evaluation criteria for the CEC 2014 special session on single objective real-parameter numerical optimization. Technical Report, 201311, Computational Intelligence Laboratory, Zhengzhou University, Nanyang Technological University (2013).
51. Awad, N. H., Ali, M. Z., Suganthan, P. N., Liang, J. J. & Qu, B. Y.: Problem definitions and evaluation criteria for the CEC2017 special session and competition on single objective real-parameter numerical optimization. Nanyang Technological University, Technical Report (2016).
52. Warchulski, E., Arabas, J. & Biedrzycki, R.: Improving the differential evolution strategy by coupling it with CMA-ES. In: GECCO 2022 Companion—Proceedings of the 2022 Genetic and Evolutionary Computation Conference (2022).
53. Su, H. et al. RIME: A physics-based optimization. *Neurocomputing*. <https://doi.org/10.1016/j.neucom.2023.02.010> (2023).
54. Zhao, W., Zhang, Z. & Wang, L. Manta ray foraging optimization: An effective bio-inspired optimizer for engineering applications. *Eng. Appl. Artif. Intell.* <https://doi.org/10.1016/j.engappai.2019.103300> (2020).
55. Xue, J. & Shen, B. Dung beetle optimizer: A new meta-heuristic algorithm for global optimization. *J. Supercomput.* <https://doi.org/10.1007/s11227-022-04959-6> (2023).
56. Li, S., Chen, H., Wang, M., Heidari, A. A. & Mirjalili, S. Slime mould algorithm: A new method for stochastic optimization. *Futur. Gener. Comput. Syst.* <https://doi.org/10.1016/j.future.2020.03.055> (2020).
57. Chopra, N. & Mohsin Ansari, M. Golden jackal optimization: A novel nature-inspired optimizer for engineering applications. *Expert Syst. Appl.* <https://doi.org/10.1016/j.eswa.2022.116924> (2022).
58. Ozkaya, B., Duman, S., Kahraman, H. T. & Guvenc, U. Optimal solution of the combined heat and power economic dispatch problem by adaptive fitness-distance balance based artificial rabbits optimization algorithm. *Expert Syst. Appl.* <https://doi.org/10.1016/j.eswa.2023.122272> (2024).
59. Yin, S., Luo, Q., Du, Y. & Zhou, Y. DTSMA: Dominant swarm with adaptive T-distribution mutation-based slime mould algorithm. *Math. Biosci. Eng.* <https://doi.org/10.3934/MBE.2022105> (2022).
60. Chen, H., Wang, Z., Jia, H., Zhou, X. & Abualigah, L. Hybrid slime mold and arithmetic optimization algorithm with random center learning and restart mutation. *Biomimetics*. <https://doi.org/10.3390/biomimetics8050396> (2023).
61. Xiao, Y. et al. IHAOAVOA: An improved hybrid aquila optimizer and African vultures optimization algorithm for global optimization problems. *Math. Biosci. Eng.* <https://doi.org/10.3934/mbe.2022512> (2022).
62. Wu, D. et al. Modified sand cat swarm optimization algorithm for solving constrained engineering optimization problems. *Mathematics*. <https://doi.org/10.3390/math10224350> (2022).
63. Wu, D., Wang, S., Liu, Q., Abualigah, L. & Jia, H. An improved teaching-learning-based optimization algorithm with reinforcement learning strategy for solving optimization problems. *Comput. Intell. Neurosci.* <https://doi.org/10.1155/2022/1535957> (2022).

Author contributions

R.W.: conceptualization, methodology, writing, data testing, software. S.Z.: conceptualization, methodology, writing, reviewing. B.J.: conceptualization, supervision, formal analysis. All authors reviewed the manuscript.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1038/s41598-024-69010-5>.

Correspondence and requests for materials should be addressed to R.W.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2024